

实验 9 处理机调度

9.1 实验目标

- (1)了解处理机调度基本原理和常见算法
- (2)在 openEuler 系统环境中分析调度算法
- (3)掌握 openEuler 系统中调度信息查看方法

9.2 预备知识

9.2.1 处理机调度概述

在多道程序系统中，内存中同时运行多道程序，它们竞争处理机资源。处理机调度指按照特定算法从就绪队列中选择一个进程，并将处理机分配给它，以实现进程并发地执行。

一般情况下，当占用处理机的进程因为某种请求得不到满足而不得不放弃 CPU 进入等待状态时，或者当时间片到，系统不得不将 CPU 分配给就绪队列中另一进程的时候，都要引起处理机调度。除此之外，进程正常结束、中断处理等也可能引起处理机的调度。因此，处理机调度是操作系统核心的重要组成部分，其主要功能如下：

- ①记录进程状态，如进程名称、指令计数器、程序状态寄存器及所有通用寄存器等信息，将这些信息记录在相应的进程控制块中；
- ②依据特定算法，决定哪个进程能获得处理机，以及占用多长时间；
- ③正在执行的进程因为时间片用完或因为某种原因不能再执行的时候，保存该进程的现场，并收回处理机。

实际上处理机调度可以分为 3 个层次：

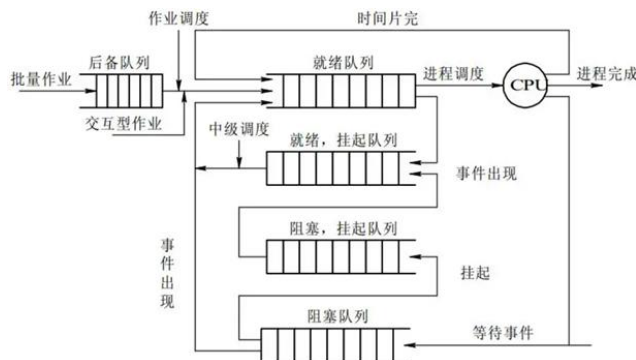


图 9.1 处理机 3 级调度示意图

(1)作业调度。又称高级调度，其主要任务是按一定的原则从外存上处于后备状态的作业中挑选一个(或多个)作业，给它(们)分配内存、输入/输出设备等必要的资源，并建立相应的进程，以使它(们)获得竞争处理机的权利。简言之，就是内存与辅存之间的调度。对于每个作业只调入一次、调出一次。多道批处理系统中大多配有作业调度，而其他系统中通常不需要配置作业调度。作业调度的执行频率较低，通常为几分钟一次。

(2)内存调度。又称中级调度。引入中级调度是为了提高内存利用率和系统吞吐量。为此，应使那些暂时不能运行的进程，调至外存等待，把此时的进程状态称为挂起状态。当它们已具备运行条件内存又稍有空闲时，由中级调度来决定，把外存上的那些已具备运行条件的就绪进程，一再重新调入内存，并修改其状态为就绪状态，挂在就绪队列上等待。

(3)进程调度。又称为低级调度，其主要任务是按照某种方法和策略从就绪队列中选取一个进程，将处理机分配给它。进程调度是操作系统中最基本的一种调度，在一般操作系统中都必须配置进程调度。进程调度的频率很高，一般几十毫秒一次。

9.2.2 常见调度算法

(1)先来先服务调度算法

先来先服务(FCFS)调度算法是一种最简单的调度算法，该调度算法既可以用于作业调度也可以用于进程调度。在作业调度中，FCFS 每次从后备作业队列中选择最先进入该队列的一个或几个作业，将它们调入内存，分配必要的资源，创建进程并放入就绪队列。在进程调度中，FCFS 每次从就绪队列中选择最先进入该队列的进程，将处理机分配给它，使之投入运行，直到完成或因某种原因而阻塞时才释放处理机。

FCFS 属于不可剥夺算法。从表面上看，它对所有作业都是公平的，不会导致作业/进程饥饿，但若一个长作业先到达系统，就会使后面许多短作业等待很长时间，因此它不能作为分时系统和实时系统的主要调度策略。它常被结合在其他调度策略中使用。例如，在使用优先级作为调度策略的系统中，往往对多个具有相同优先级的进程按 FCFS 原则处理。FCFS 调度算法的特点是算法简单，但效率低；对长作业比较有利，但对短作业不利；有利于 CPU 繁忙型作业，而不

利于 I/O 繁忙型作业。

(2)短作业优先算法

短作业优先(SJF)调度算法是从后备队列中选择一个或若干个估计运行时间最短的作业，将它们调入内存运行。而**短进程优先(SPF)**调度算法，则是从就绪队列中选择一个估计运行时间最短的进程，将处理机分配给它，使之立即执行，直到完成或发生某事件而阻塞时，才释放处理机。

SJF 存在不容忽视的缺点：1)该算法对长作业不利，会导致作业/进程饥饿。2)该算法完全未考虑作业的紧迫程度，因而不能保证紧迫性作业会被及时处理，非抢占。3)由于作业的长短只是根据用户所提供的估计执行时间而定的，而用户又可能会有意或无意地缩短其作业的估计运行时间，致使该算法不一定能真正做到短作业优先调度。注意，SJF 的平均等待时间、平均周转时间最少。

(3)优先级调度算法

优先级调度算法又称优先权调度算法，该算法既可以用于作业调度，也可以用于进程调度。该算法中的优先级用于描述作业运行的紧迫程度。在作业调度中，优先级调度算法每次从后备作业队列中选择优先级最高的一个或几个作业，将它们调入内存，分配必要的资源，创建进程并放入就绪队列。在进程调度中，优先级调度算法每次从就绪队列中选择优先级最高的进程，将处理机分配给它，使之投入运行。根据新的更高优先级进程能否抢占正在执行的进程，可将该调度算法分为剥夺式和非剥夺式优先级调度算法。

根据进程创建后其优先级是否可以改变，可以将进程优先级分为以下两种：

①静态优先级。即优先级是在创建进程时确定的，且在进程的整个运行期间保持不变。确定静态优先级的主要依据有进程类型、进程对资源的要求、用户要求；

②动态优先级。即在进程运行过程中，根据进程情况的变化动态调整优先级。动态调整优先级的主要依据为进程占有 CPU 时间的长短、就绪进程等待 CPU 时间的长短。

(4)时间片轮转调度算法

时间片轮转调度算法主要适用于分时系统。在这种算法中，系统将所有就绪进程按到达时间的先后次序排成一个队列，进程调度程序总是选择就绪队列中第

一个进程执行，即先来先服务的原则，但仅能运行一个时间片，如 100ms。在使用完一个时间片后，即使进程并未完成其运行，它也必须释放出(被剥夺)处理机给下一个就绪的进程，而被剥夺的进程返回到就绪队列的末尾重新排队，等候再次运行。在时间片轮转调度算法中，时间片的大小对系统性能的影响很大。如果时间片足够大，以至于所有进程都能在一个时间片内执行完毕，则时间片轮转调度算法就退化为先来先服务调度算法。如果时间片很小，那么处理机将在进程间过于频繁切换，使处理机的开销增大，而真正用于运行用户进程的时间将减少。因此时间片的大小应选择适当。时间片的长短通常由以下因素确定:系统的响应时间、就绪队列中的进程数目和系统的处理能力。

9.3 实验步骤

9.3.1 处理机调度算法模拟

在该实验中，需设计一个按时间片轮转调度的算法，具体内容如下：

- ① 假设系统内有 3 个进程，每个进程由一个进程控制块(PCB)来管理。进程控制块内容如下图所示：

进程名
链接指针
到达时间
估计运行时间
进程状态

图 9.2 进程控制块内容示意图

上图字段解释：**进程名**，即进程标识；**链接指针**，按进程到达时间处于就绪状态的进程组成一个就绪队列。指针指出下一个到达进程的进程控制块。最后一个进程的链接指针为 NULL；**到达时间**，进程创建时的系统时间或由用户指定。调度时总是选择到达时间最早的进程；**估计运行时间**，设计者任意指定的一个时间值；**进程状态**，为简单起见，假定进程只有两种状态(就绪和完成)。进程一创建就处于就绪状态(R)，进程运行结束置为完成状态。

- ② 系统设置一个队头和一个队尾指针，分别指向队列的第一个和最后一个进程；
- ③ 为每个进程设置必要的运行时间和到达时间；

- ④ 按进程到达的先后次序排成一个循环队列；
- ⑤ 处理机调度时，开始选择队首第一个进程运行，另外再设一个当前运行进程指针，指向当前正运行的进程；
- ⑥ 实验是模拟进程调度，**实际执行**：1)运行时间减 1 和输出当前运行进程名；2)进程运行 1 次后，应将当前运行进程指针下移一个位置，指向下一个进程。同时，还应判断进程的剩余运行时间是否为 0。若不为 0，则等待下一轮调度；若为 0，则状态改为完成状态，并撤出就绪队列；
- ⑦ 若就绪队列不空，重复上述第⑥步。
- ⑧ 程序中应有显示和打印语句，显示每次选中进程名字和队列变化情况。

具体实验步骤如下：

(1)创建 C 语言代码文件，命名为 **schedule.c**，并使用 **vi** 命令打开创建的文件，命令如下：

```
[root@localhost ~]# touch schedule.c
[root@localhost ~]# vi schedule.c
```

(2)按“**i**”键进入 **insert** 模式，在 **schedule.c** 文件中输入如下代码，按“**Esc**”退出 **insert** 模式，输入“**wq**”保存。

```
#define N 20
#include <stdio.h>
#include <stdlib.h>
typedef struct pcb{ /* 进程控制块定义 */
    char pname[N];
    int runtime;
    int arrivetime;
    char state;
    struct pcb *next;
}PCB;
PCB head_input; //就绪队列头指针
PCB head_run;   //运行队列头指针
static char R='r',C='c';
unsigned long current; //记录系统当前时间
void inputprocess(); /* 建立进程函数 */
int readyprocess(); /* 建立就绪队列函数 */
int readydata(); /* 判断进程是否就绪函数 */
int runprocess(); /* 运行进程函数 */
int readyprocess() {
    while(1) {
        if(readydata()==0) return 1;
```

```

        else runprocess();
    }
}

int readydata() { //判断就绪队列是否为空
    PCB *p1,*p2,*p3;
    if(head_input.next==NULL) {
        if(head_run.next==NULL) return 0;
        else return 1;
    }
    p1 = head_run.next;
    p2 = &head_run;
    while (p1 != NULL) {
        p2 = p1;
        p1 = p2->next;
    }
    p1 = p2;
    p3 = head_input.next;
    p2 = &head_input;
    while (p3 != NULL) {
        if (( (unsigned long)p3->arrivetime <= current) && (p3->state == R)) {
            printf("Time slice is %8ld( time %4ld); Process %s start.\n",
                current, (current+500)/1000, p3->pname);

            p2->next = p3->next;
            p3->next = p1->next;
            p1->next = p3;
            p1 = p3;
            p3 = p2;
        }
        p2 = p3;
        p3 = p3->next;
    }
    return 1;
}

int runprocess() {
    PCB *p1,*p2;
    if(head_run.next == NULL) {
        current++;
        return 1;
    } else {
        p1 = head_run.next;
        p2 = &head_run;
        while(p1 != NULL) {
            p1->runtime--;
            current++;

```

```

        if(p1->runtime <= 0) {
            printf("Time slice is %8ld( time %4ld); Process %s end.\n",
                current, (current+500)/1000, p1->pname);
            p1->state = C;
            p2->next = p1->next;
            free(p1);
            p1 = p2->next;
        } else {
            p2 = p1;
            p1 = p2->next;
        }
    }
    return 1;
}

void inputprocess() {
    PCB *p1,*p2;
    int i,num;
    unsigned long max = 0;
    printf("How many processes do you want to run:");
    scanf("%d",&num);
    p2 = &head_input;
    for (i=0; i<num; i++) {
        p1 = (PCB *)malloc(sizeof(PCB));
        p2->next = p1;
        printf("No.%3d process input pname:",i+1);
        scanf("%s",p1->pname);
        printf(" runtime:");
        scanf("%d",&(p1->runtime));
        printf(" arrivetime: ");
        scanf("%d",&(p1->arrivetime));
        p1->runtime = (p1->runtime)*1000;
        p1->arrivetime = (p1->arrivetime)*1000;
        p1->state = R;
        if(((unsigned long)(p1->arrivetime) > max)
            max = p1->arrivetime;
        p2 = p1;
    }
    p2->next = NULL;
}

void main() {
    printf("\ntime 1 = 1000 time slice\n");
    current = 0;
    inputprocess();
}

```

```
    readyprocess();  
    getc(stdin);  
}
```

(2)编译 **schedule.c**, 生成可执行文件 **schedule** 并执行, 具体如下:

```
[root@localhost ~]# gcc schedule.c -o schedule
```

```
[root@localhost ~]# ./schedule
```

可得到如下的输出结果:

```
time 1 = 1000 time slice  
How many processes do you want to run:3  
No. 1 process input pname:a  
    runtime:10  
    arrivetime: 12  
No. 2 process input pname:b  
    runtime:5  
    arrivetime: 15  
No. 3 process input pname:c  
    runtime:6  
    arrivetime: 20  
Time slice is 12000( time 12); Process a start.  
Time slice is 15000( time 15); Process b start.  
Time slice is 20000( time 20); Process c start.  
Time slice is 27499( time 27); Process b end.  
Time slice is 31499( time 31); Process a end.  
Time slice is 33000( time 33); Process c end.
```

需要注意的是, 该程序总共有 3 个进程, 对应的 3 个进程块信息需在运行程序后自行键入设定, 最终程序会显示 3 个进程全部运行完的时间情况。

9.3.2 内核调度算法探究

在 Linux 内核中, 实时进程总是比普通进程优先级要高, 实时进程调度是由 RT(Real Time)调度器管理(可阅读 Linux 内核 RT 实现代码¹), 而普通进程由 CFS 调度器来管理(可阅读 Linux 内核 CFS 实现代码²)。

CFS(Completely Fair Scheduler)是 Linux 内置(也是目前默认)的一个内核调度器。如其名字, CFS 实现了所谓的“完全公平”调度, 将 CPU 资源均匀地分配给各进程(在内核代码中称为“任务”, Task), 通过限制当前 Task 运行时间来实现公平性。简单来说, 若一台机器有一个 CPU 多个(计算密集型)进程, 那采用 CFS 调度器时, 2 个进程各占 50% CPU 时间, 4 个进程各占 25% CPU 时间。CFS

¹ <https://elixir.bootlin.com/linux/v5.15/source/kernel/sched/rt.c>

² <https://elixir.bootlin.com/linux/v5.15/source/kernel/sched/fair.c>

使用虚拟时间 `vruntime`(单调递增)来决定运行的 Task，每次选取 `vruntime` 最小的 Task 进行执行，并用 `rbtree` 管理调度实体。

有关 RT 和 CFS 内容的详细介绍可参考网络资料³，此处不深入讨论。在实验前，请先查资料熟悉 `top`、`ps`、`/proc/cpuinfo`、`/proc/PID/sched`、`/proc/PID/status`、`/proc/PID/stat`、`/proc/sched_debug`、`/proc/schedstat` 等。此外，普通进程的 `nice` 值在 -20 到 19 之间，默认优先级为 120，也即普通进程优先级可在 100-139 间变化，0-99 是实时进程。优先级值越小，则优先级越高⁴。

CFS 调度实验过程：

(1)在一个空闲单核(启动前将虚拟机的 CPU 核数调整为 1)openEuler 系统上用 `nice` 命令调整两个进程的优先级。**要求：**使两个进程各使用约 1/5 和 4/5 的 CPU 资源。用 `top` 和 `/proc/PID/sched` 展示各进程使用的调度策略以及调整前后的优先级变化，用 `top` 命令展示 CPU 资源分配的效果。

①为保证进程能一直运行，编写如下的死循环代码，并将其编译成文件名为 `proc1` 和 `proc2` 的可执行文件，然后以后台进程(&)形式运行两个程序：

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    while(1);
    return 0
}
```

②使用 `top` 命令查看 `proc1` 和 `proc2` 两个程序对应进程占用 CPU 资源的情况；

③使用 `cat /proc/<PID>/sched` 查看两个进程的实际 **prio** 值(正常都是默认值 120)，此时进程的 NI 值为 0。

④使用 **renice** 命令(如 **renice -17 <PID>**)重新设置两个进程的优先级(自行调整，如 -17 和 -11，具体原则如下描述)，使两个进程对 CPU 的占用约为 4/5 和 1/5，并使用 `top` 命令查看 `proc1` 和 `proc2` 两个程序对应进程占用 CPU 资源的情况；

CFS 实现不同 `nice` 值 CPU 时间比例在内核中是按照“每差一级 CPU 占用时间差 10%左右”的原则来设定。大概意思是说，若两个 `nice` 值为 0 的进程同时占用 CPU，则它们应该各占 50%的 CPU。若将其中一个进程 `nice` 值设为 1，此时应保证优先级高的进程比低的多占用 10%的 CPU，即 `nice` 为 0 的占

³ <https://s3.shizhz.me/linux-sched/rt-sched>

⁴ <https://elixir.bootlin.com/linux/v5.15/source/include/linux/sched/prio.h>

55%，nice 为 1 的占 45%，占用 CPU 时间的比例为 55:45，约为 1.25。根据该原则，内核对 40 个 nice 值做了时间计算比例的对应关系，它在内核中以一个数组存在：

```
static const prio_to_weight[40]={  
    /* -20 */88761,71755,56483,46273,36291,  
    /* -15 */29154,23254,18705,14949,11916,  
    /* -10 */9548,7620,6100,4904,3906,  
    /* -5 */3121,2501,1991,1586,1277,  
    /* 0 */1024,820,655,526,423,  
    /* 5 */335,272,215,172,137,  
    /* 10 */110,87,70,56,45,  
    /* 15 */36,29,23,18,15,  
};
```

(2) 在一个空闲双核 openEuler 系统中启动 4 个一直就绪不阻塞的进程 proc1/proc2/proc3/proc4(后台运行，代码同前 1 个实验)。**要求**，使得 proc1/proc3 在第 1 个核上运行且各占 50% 的 CPU 资源，proc2/proc4 在另一个核上运行且各占 30% 和 70% CPU 资源。

① 查看并记录 `/proc/cpuinfo` 显示的系统核数；

② 查看并记录各进程在各核上的绑定情况(`pidstat -p <PID> -t 1`)；

③ 查看并记录 `top` 命令给出的 CPU 资源分配情况；

④ 查看各进程亲和力掩码(affinity mask)(`taskset -p <PID>`)，数字 3 表示二进制 $(11)_2$ ，即进程可使用两个 CPU。使用 `taskset` 的 `-pc` 参数，可将进程绑定至指定 CPU，如 `taskset -pc <CPU 编号, 0/1> <PID>`；

⑤ 使用 `renice` 命令将 proc3 的优先级设置为 5，`top` 中查看 CPU 优先级，proc4 和 proc3 各占 CPU 1 约 70% 和 30%，proc2 和 proc1 各占 CPU 0 约 50%；

⑥ 使用 `head -n 6 /proc/<PID>/sched` 打印迁移次数，此迁移只发生在绑定 CPU 核前；

⑦ 运行第 5 个不阻塞进程，用 `top` 查看并记录负载均衡现象，用 `/proc/PID/sched` 查看并记录各进程在核间的迁移次数。可以看到，第 5 个进程抢占了 CPU 资源。proc4 和 proc3 的 CPU 占用都有所下降，而 proc2 和 proc1 也有所下降。

⑧ 使用 `head -n 6 /proc/<PID>/sched` 打印 5 个进程迁移次数，可以看到，未绑定 CPU 核的第 5 个进程迁移数不断上升，而已绑定核心的进程不在上升。

四、实验讨论

(1)在前述实验中，分析了 CFS，读者可以使用类似方式分析 **RT 调度**，有关 RT 的实现请自行调研。

(2)修改 nice 值可以改变进程优先级，是否可以将普通进程直接改为实时进程？

(3)在进程调度模拟部分仅给出了 RR 的视线，请根据 FCFS/SJF 等算法思想模拟进程调度。